Programa:	1 Click & Lock 2.8	
Protección:	Cinderella 30 días	
Objetivos:	Encontrar un serial válido	
URL descarga:	http://www.softstack.com	
Dificultad:	Muy fácil	
Herramientas:	OllyDbg 1.10	
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos]	Fecha: 05-09-2005

Hola una vez más a todos!!

Ya estamos aquí nuevamente, hoy dedicaremos un rato a ver como se protegen algunas aplicaciones comerciales mediante un suave algoritmo name/serial del que encontraremos un serial válido, hay que resaltar que estos esquemas son muy típicos dentro del mundillo Shareware y nos los podemos encontrar muy a menudo, es por esto que hoy veremos como deshacernos de ellos con mucha facilidad.

No perdamos más tiempre pues y pasemos al ataque ;-)

Primer Acercamiento… y definitivo!!

En primer lugar, siempre que encuentro una protección de estas yo personalmente hago lo mismo, primero sin Olly ni nada ejecutoa el programa normalmente y la examino, miro posibles funciones deshabilitadas, si tiene limitación temporal, si muestra alguna nag, cuantas muestra si es así, etc., todo esto lo apunto aparte para después guiarme, esto lógicamente no es obligatorio ni mucho menos pero sí es bueno hacerlo, es más, el recopilar información sí es obligatorio y fundamental a la vez, contra más datos obtengamos de entrada más numerosos serán nuestros ataques esto es así. Ahora una vez tenemos los datos suficientes sí cargamos el programa en Olly. En este caso como vimos que mostraba un mensaje de error atacaremos por él, pongamos en el commadbar: BP MessageBoxA + Enter y damos RUN, el programa carga, ponemos un serial y aceptamos, aparece el mensaje y Olly no salta ¿? No importa, cuando esté el mensaje de error en pantalla pondremos BP DestroyWindow y aceptamos el mensaje, ahora Olly si romperá:

He de decir que esta API nos dejará 5 ó 6 llamadas por debajo del call buscado, casi siempre es así, almenos yo nunca me he encontrado casos distintos a este. Según lo dicho cuando rompa Olly quitamos el BP para que no nos moleste y traceamos superando los RET's que vayamos encontrando hasta salir en este lugar tan preciado, jejeje..

```
Address   Hex dump     Disassembly                  Comment
004EED37  .  E8 34D9FBFF  CALL 1cl.004AC670          Call verificador..!!
004EED3C  .  84C0         TEST AL,AL
004EED3E  .v 74 21        JE SHORT 1cl.004EED61      Salto decisivo
004EED40  .  8B45 FC      MOV EAX,DWORD PTR SS:[EBP-4]
004EED43  .  E8 54C0FAFF  CALL 1cl.0049AD9C
004EED48  .  6A 00        PUSH 0                     Arg1 = 00000000
004EED4A  .  66:8B0D A8ED4 MOV CX,WORD PTR DS:[4EEDA8]
004EED51  .  B2 02        MOV DL,2
004EED53  .  B8 B4ED4E00  MOV EAX,1cl.004EEDB4       ASCII "Registration key is ok"
004EED58  .  E8 475BF4FF  CALL 1cl.004348A4          1cl.004348A4
004EED5D  .  B3 01        MOV BL,1
004EED5F  .v EB 23        JMP SHORT 1cl.004EED84
004EED61  >  6A 00        PUSH 0                     Arg1 = 00000000
004EED63  .  66:8B0D A8ED4 MOV CX,WORD PTR DS:[4EEDA8]
004EED6A  .  B2 01        MOV DL,1
004EED6C  .  B8 D4ED4E00  MOV EAX,1cl.004EEDD4       ASCII "Registration key is not valid"
004EED71  .  E8 2E5BF4FF  CALL 1cl.004348A4          1cl.004348A4
004EED76  .  A1 B0284F00  MOV EAX,DWORD PTR DS:[4F28B0]  Aparecemos aqui
004EED7B  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
004EED7D  .  C680 C9030000 MOV BYTE PTR DS:[EAX+3C9],1
```

Vaya donde hemos salido, de poco les sirvió no utilizar la api MessageBoxA, pues le hemos descubierto el escondrijo igual, a partir de aquí seguimos como siempre, pongamos un BP en el call que decide que mensaje se va a mostrar como se ve en la imagoen y reiniciemos Olly con Ctrl+F2, volvemos a poner un serial y ahora paramos allí en el call, entremos con F7 a ver que va haciendo:

```
004EED37 |.  E8 34D9FBFF     CALL 1cl.004AC670                 ; Rutina de verificación -> Entremos…
004EED3C |.  84C0             TEST AL,AL                        ; en AL estará el valor de retorno
004EED3E |.  74 21            JE SHORT 1cl.004EED61             ; si quedó AL==0 fue serial no válido
004EED40 |.  8B45 FC          MOV EAX,DWORD PTR SS:[EBP-4]      ; Salto decisivo
```

dentro del call encontramos la primera restricción qu es esta:

```
004AC6A4 |.  E8 0F84F5FF      CALL 1cl.00404AB8                 ; Deja en EAX el número de dígitos del serial.
004AC6A9 |.  83F8 10          CMP EAX,10                        ; si no es 16 vamos a error (hablaamos en decimal)
004AC6AC |.  0F85 A4000000    JNZ 1cl.004AC756                  ; Salto a Invalid_key..!!
```

Como nuestro serial no tiene la lingitud correcta, empezamos de nuevo pero poniendo un serial de 16 dígitos ahora y seguimos traceando la rutina a partir de aquí ya que ahora superaremos la restricción sin problemas, un poco después se mete en un loop en el que verifica a mi parecer que ningún carácter del serial se repita más de 5 veces, si esto no ocurre llegamos al último call donde todo se decide:

```
004AC74B |.  E8 B0FDFFFF      CALL 1cl.004AC500                 ; Call verificador
004AC750 |.  84C0             TEST AL,AL                        ; Debe ser AL==1
004AC752 |.  74 02            JE SHORT 1cl.004AC756             ; No debe saltar
004AC754 |.  B3 01            MOV BL,1                          ; Flag de éxito activado!
004AC756 |>  33C0             XOR EAX,EAX
```

dentro de este call encontramos:

```
004AC574  |.  A1 C02C4F00          MOV EAX,DWORD PTR DS:[4F2CC0]
004AC579  |.  3B30                 CMP ESI,DWORD PTR DS:[EAX]
004AC57B  |.  0F85 BA000000        JNZ 1cl.004AC63B              ; Salto a Invalid_key..!!
```

Curiosamente ESI tiene un valor fijo y el que tiene [EAX] va variando según el serial introducido, de esto sabemoe que verifica parte del serial, si nos fijamos más vemos que la suma de los cuatro primeros dígitos debe ser igual a 17h = 23d, por ejemplo: **4937**

Esto lo podemos ver bien siendo observadores o bien siguiendo el código, de ambos modos se obtiene el resultado, del segundo modo es mucho más lógico y coherente, a ser posible siempre intentaremos obtener los datos del código tal cual se ve y deducir lo que no se vea, por el primer método se aprecia también si vamos variando el serial, por ejemplo:

1234567891234650
pide: 17h resulta: 0Ah

1284567891234650
pide: 17h resulta: 0Fh

Así no es difícil darse cuenta que los primeros dígitos tienen que ver, si modificamos los restantes no influye, o sea que se centra en los cuatro primeros, concretamente compara el resultado siempre con 17h pasado a decimal sería 23, busquemos pues 4 números que sumen 23, fácil por ejemplo 4937, nuestro serial comenzará de esta manera, sigamos, ahora pondremos como serial:

4937567891234650

y aceptamos, al llegar a la comparación donde antes nos tiraba vemos que no se queja, le gustó el cambio, jejeje.. vamos bien, peeeeeeeeero tiene más de este tipo, siguiendo el segundo método de estudiar el código vemos como va tomando los caracteres de este modo:

```
004AC584  |.  8A53 04      MOV DL,BYTE PTR DS:[EBX+4]
004AC599  |.  8A53 07      MOV DL,BYTE PTR DS:[EBX+7]
004AC5AE  |.  8A53 0A      MOV DL,BYTE PTR DS:[EBX+A]
004AC5C3  |.  8A53 0D      MOV DL,BYTE PTR DS:[EBX+D]
```

Teniendo en cuenta que EBX tiene un puntero a la cadena sabemos que se refiere a los dígitos 5, 8, 11 y 14, los suma, este resultado al final lo compara con 1Eh, o sea 30 para nosotros, busquemos pues un serial cuyos dígitos que ocupen estas posiciones sumen 30d, fácil también:

4937901891738658

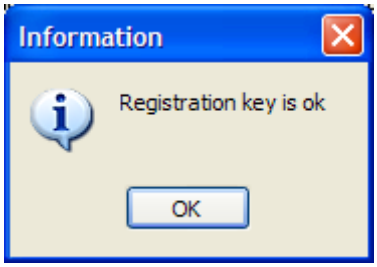
Al llega a la comparación en :4AC5DC donde nos echaba vemos que pasamos sin mayores problemas, sigamos pues que nos queda ya poco, una última ráfaga cuyos dígitos implicados son:

```
004AC5E1  |.  8A53 06      MOV DL,BYTE PTR DS:[EBX+6]
004AC5F6  |.  8A53 09      MOV DL,BYTE PTR DS:[EBX+9]
004AC60B  |.  8A53 0C      MOV DL,BYTE PTR DS:[EBX+C]
004AC620  |.  8A53 0F      MOV DL,BYTE PTR DS:[EBX+F]
```

O sea, los de las posiciones 7, 10, 13 y 16 en concreto deben sumar 9h o sea también 9 decimal, pues vamos allá, pongamos:

4937901890730658

Lo introducimos como serial y… ZASHHHHHHHH!!



Registro aceptado, lo hemos pillado, si a alguien le interesa saber dónde esconde los datos de registro lo puede saber poniendo un par de BP's en RegOpenKeyA y RegOpenKeyExA, damos al About y Olly salta:

```
0012F1FC  0043C9A3  CALL to RegOpenKeyExA from 1cl.0043C99E
0012F200  80000002  hKey = HKEY_LOCAL_MACHINE
0012F204  00A8F8E8  Subkey = "Software\1 Click & Lock\Settings"
0012F208  00000000  Reserved = 0
0012F20C  00020019  Access = KEY_READ
0012F210  0012F230  pHandle = 0012F230
```

Ahí tenemos el nombre de la clave que va a buscar al registro y está en HKLM, opdemos ir ya a esa clave y mirar los valores que haya, pero si traceamos un poquitín más salimos en una zona interesante:

```
0049AEF9  |.  BA 68AF4900          MOV EDX,1cl.0049AF68          ;  ASCII "Inf2"
0049AEFE  |.  8BC3                 MOV EAX,EBX
0049AF00  |.  E8 6720FAFF          CALL 1cl.0043CF6C
```

Ese ***Inf2*** es muy indicativo, es evidente que allí tiene guardado el serial, vamos a mirarla y la encontramos pero está cifrada ya no se ve el serial tal cual lo pusimos, lo que hace a continuación es meterse en algunos CALL donde lo descifra y finalmente salimos aquí:

```
004EDF46  .  8D45 FC        LEA EAX,DWORD PTR SS:[EBP-4]
004EDF49  .  E8 42CFFAFF    CALL 1cl.0049AE90            ; Dentro está todo el lío
004EDF4E  .  8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4] ; En EAX el serial desencriptado
004EDF51  .  E8 1AE7FBFF    CALL 1cl.004AC670            ; Lo verifica => si bueno retorna EAX==1
```

Cuando retornamos aquí ya ha desencriptado el serial y lo carga en ***EAX*** a la salida de la llmada, lueog lo va a verificar en ***:4EDF51***, si es bueno retonará ***EAX==1*** como apuntan los comentarios, esto está visto porque es la misma llamada que antes estudiamos.

Con esto sólo quería mostrar a aquellas personas que realmente empiezan en esto y dan sus primeros pasos cómo se puede localizar un dato (serial o lo que sea) en el registro cuando suponemos que el programa lo debe guardar allí, por eso hemos utilizado las apis ***RegOpenKeyA*** y ***RegOpenKeyExA*** que se utilizan para detener la ejecución cuando el programa intenta acceder a una clave del registro del sistema (el ejecutable que nos permite editarlo se llama ***regedit.exe***)

Tenemos entonces que un serial válido entre todos los posibles sería:

Serial válido: 4937901890730658

Con esto pienso que por hoy está bien, los objetivos han sido cubiertos, hemos obtenido un serial válido para el programa y sin modificar un solo byte, esta siempre será, sin duda, la mejor solución.

Un saludo a todo ***CracksLatinos***, pronto seguiremos con nuevos estudios e investigaciones, para cualquier cosa me encontraréis en:

syxe00@yahoo.es
syxe00@hotmail.com